

Sender Algorithm Customization

Draft Proposal

Document #: P2999R3
Date: 2023-12-12
Project: Programming Language C++
Audience: LEWG Library Evolution
Reply-to: Eric Niebler
<eric.niebler@gmail.com>

Contents

- 1 Introduction
 - 1.1 The Issue
 - 1.2 Revision history
 - 1.2.1 R2
 - 1.2.2 R1
 - 1.2.3 R0
- 2 Proposed Design
 - 2.1 Features and rationale
 - 2.1.1 Dispatching via execution domain tags
 - 2.1.2 Late (sender/receiver connection-time) customization
 - 2.1.3 Early (sender construction-time) customization
 - 2.1.4 Decomposable senders
 - 2.2 Why have both early and late customization?
 - 2.3 An example of early and late customizations
 - 2.4 Summary of proposed changes
- 3 Implementation Experience
- 4 Proposed Wording
- 5 References

§ 1 Introduction

This paper proposes some design changes to P2300 to address some shortcomings in how algorithm customizations are found.

§ 1.1 The Issue

The essence of the issue is this:

Many senders do not know on what execution context they will complete, so using solely that information to find customizations (as P2300R7 does) is unsatisfactory.

In [P2300R7], the sender algorithms (then, let_value, etc) are customization point objects that internally dispatch via tag_invoke to the correct algorithm implementation. Each algorithm has a default implementation that is used if no custom implementation is found.

Custom implementations of sender algorithms are found by asking the predecessor sender for its completion scheduler and using the scheduler as a tag for the purpose of tag dispatching. A *completion scheduler* is a scheduler that refers to the execution context on which that sender will complete.

A typical sender algorithm like then might be implemented as follows:

```
/// @brief A helper concept for testing whether an algorithm customization
/// exists
template <class AlgoTag, class SetTag, class Sender, class... Args>
concept has-customization =
    requires (Sender snd, Args... args) {
        tag_invoke(AlgoTag(),
            get_completion_scheduler<SetTag>(get_env(snd)),
            std::forward<Sender>(snd),
            std::forward<Args>(args)...);
    };

/// @brief The tag type and the customization point object type for the
/// `then` sender algorithm
struct then_t {
    template <sender Sender, class Fun>
        requires /* requirements here */
        auto operator()(Sender&& snd, Fun fun) const
        {
            // If the predecessor sender has a completion scheduler, and if we can use
            // the completion scheduler to find a custom implementation for the `then`
            // algorithm, dispatch to that. Otherwise, dispatch to the default `then`
            // implementation.
            if constexpr (has-customization<then_t, set_value_t, Sender, Fun>)
            {
                auto&& env = get_env(snd);
                return tag_invoke(*this,
                    get_completion_scheduler<set_value_t>(env),
                    std::forward<Sender>(snd),
                    std::move(fun));
            }
            else
            {
                return then_sender<Sender, Fun>(std::forward<Sender>(snd), std::move(fun));
            }
        }
};

inline constexpr then_t then {};
```

This scheme has a number of shortcomings:

1. A simple sender like just(42) does not know its completion scheduler. It completes on the execution context on which it is started. That is not known at the time the sender is constructed, which is when we are looking for customizations.
2. For a sender like on(sch, then(just(), fun)), the nested then sender is constructed before we have specified the scheduler, but we need the scheduler to dispatch to the correct customization of then. How?

3. A composite sender like `when_all(snd1, snd2)` cannot know its completion scheduler in the general case. Even if `snd1` and `snd2` both know their completion schedulers – say, `sched1` and `sched2` respectively – the `when_all` sender can complete on *either* `sched1` or `sched2` depending on which of `snd1` and `snd2` completes last. That is a dynamic property of the program’s execution, not suitable for finding an algorithm customization.

In cases (1) and (2), the issue is that the information necessary to find the correct algorithm implementation is not available at the time we look for customizations. In case (3), the issue is that the algorithm semantics make it impossible to know statically to what algorithm customization scheme to dispatch.

The issue described in (2) above is particularly pernicious. Consider these two programs (where `ex::` is a namespace alias for `std::execution`); the differences are highlighted:

Table 1: Algorithms customizations are found or not depending on subtle differences

Good	Bad
<pre> // Describe some bulk work on a given // scheduler auto work(ex::scheduler auto sch, auto data) { return ex::transfer_just(sch, data) ex::bulk(data.size(), [](int i, auto& data) { ++data[i]; }); } my::thread_pool_scheduler thread_pool = /*...*/; ex::sender auto task = work(thread_pool, data); // Execute the work std::this_thread::sync_wait(std::move(task)); </pre>	<pre> // Describe some bulk work auto work(auto data) { return ex::just(data) ex::bulk(data.size(), [](int i, auto& data) { ++data[i]; }); } my::thread_pool_scheduler thread_pool = /*...*/; ex::sender auto task = work(data); // Execute the bulk work on a thread pool std::this_thread::sync_wait(ex::on(thread_pool, std::move(task))); </pre>

These two programs *should* be equivalent, but they are not. The author of the `thread_pool_scheduler` gave it a custom bulk implementation by defining:

```

namespace my {
    // customization of the bulk algorithm for the thread_pool_scheduler:
    template <ex::sender Sender, std::integral Shape, class Function>
    auto tag_invoke(ex::bulk_t,
        thread_pool_scheduler sch,
        Sender&& snd,
        Shape shape,
        Function fun) {
        /*
         * Do bulk work in parallel
         * ...
         */
    }
}

```

This overload is found only when the bulk sender’s predecessor completes on a `thread_pool_scheduler`, which is the case for the code on the left.

In the code to the right, however, the predecessor of the bulk operation is `just(data)`, a sender that does not know where it will complete. As a result, the above customization of the bulk algorithm will not be found, and the bulk operation will execute serially on a single thread in the thread pool. That’s almost certainly *not* what the programmer intended.

This is clearly broken and badly in need of fixing.

Note: On the need for async algorithms customization

It is worth asking why async algorithms need customization at all. After all, the classic STL algorithms need no customization; they dispatch using a fixed concept hierarchy to a closed set of possible implementations.

The reason is because of the open and continually evolving nature of execution contexts. There is little hope of capturing every salient attribute of every interesting execution model – CPUs, GPUs, FPGAs, etc., past, present, and future – in a fixed ontology around which we can build named concepts and immutable basis operations. Instead we do the best we can and then hedge against the future by making the algorithms customizable. For example, say we add an algorithm `std::par_algo`, but we allow that there may be an accelerator “out there” that may do `par_algo` more efficiently than the standard one, so we make `par_algo` customizable.

§ 1.2 Revision history

§ 1.2.1 R2

- Add “[An example of early and late customizations](#)”
- Add a discussion of the [reasons to keep eager customization](#)
- Fix the computation of the domain at connect-time
- Add a requirement on senders that their completion schedulers all share a domain
- `stdexec::transform_sender` recurses when necessary
- remove no-longer-necessary exposition-only *make-transformer-fn* and all uses

§ 1.2.2 R1

- Proposed wording added
- Add comparison tables

§ 1.2.3 R0

- Initial revision

§ 2 Proposed Design

§ 2.1 Features and rationale

This section describes at a high level the salient features of the proposed design for sender algorithm customization, and their rationale. But in a nutshell, the basic idea is as follows:

For every invocation of a sender algorithm, the implementation looks for a customization *twice*: once immediately while the algorithm is constructing a sender to return, and once later *when the resulting sender is connect-ed with a receiver*.

It is the second look-up that is new. By looking for a customization at connect time, the dispatching logic is informed both by information from the predecessor sender(s) as well as from the receiver. It is the receiver that has information about the environment of the currently executing asynchronous operation, information that is key to picking the right customization in the cases we looked at above.

§ 2.1.1 Dispatching via execution domain tags

As described above, the `when_all` sender doesn't know its completion scheduler, so we cannot use the completion scheduler to find the `when_all` customization. Even if all its child senders advertise completion schedulers with the same type – say, `static_thread_pool` – `when_all` itself can't advertise a completion scheduler because it doesn't know that they are all the *same* `static_thread_pool`.

In the case just described, consider that we can know the completion scheduler's *type* but not its value. So at the very least, we need to add a query about the type of the scheduler apart from its value, for times when we know one but not the other.

Once we have done that, further generalizing the query from a scheduler type to an abstract tag type is a short hop. We call this abstract tag type an *execution domain*. Several different scheduler types may all want to use the same set of algorithm implementations; those schedulers can all use the same execution domain type for the purpose of dispatching.

If `when_all`'s child senders all share an execution domain, we know the execution domain of the `when_all` sender itself even if we don't know which scheduler it will complete on. But that no longer prevents us from dispatching to the correct implementation.

This paper proposes the addition of a forwarding `get_domain` query in the `std::execution` namespace, and that the domain is used together with the algorithm tag to dispatch to the correct algorithm implementation.

Additionally, we proposed that the `when_all` algorithm only accepts a set of senders when they all share a common domain. Likewise for `let_value` and `let_error`, we require that there is only one possible domain on which their senders may complete.

§ 2.1.2 Late (sender/receiver connection-time) customization

As described above, the sender algorithm customization points don't have all the information they need to dispatch to the correct algorithm implementation in all cases. The solution is to look again for a customization when all the information is available. That happens when the sender is connect-ed to a receiver.

This paper proposes the addition of a `transform_sender` function that is called by the `connect` customization point to transform a sender prior to connecting it with the receiver. The correct sender transformation is found using a property read from the receiver's environment.

The following comparison table shows how we propose to change the `connect` customization point (changes highlighted):

Table 2: The addition of `transform_sender` to `connect`

Before	After
<pre>struct connect_t { template <receiver Receiver, sender_in<env_of_t<Receiver>> Sender> requires /* ... */ auto operator()(Sender&& snd, Receiver&& rcv) const {</pre>	<pre>struct connect_t { template <receiver Receiver, sender_in<env_of_t<Receiver>> Sender> requires /* ... */ auto operator()(Sender&& snd, Receiver&& rcv) const {</pre>

Before	After
<pre> // First, look for a customization of tag_invoke: if constexpr (tag_invocable<connect_t, Sender, Receiver>) { return tag_invoke(*this, std::forward<Sender>(snd), std::forward<Receiver>(rcv)); } // Next, see if the sender is co_await-able: else if constexpr (is-await- connectable<Sender, Receiver>) { /* ... */ } } }; </pre>	<pre> // Apply any sender transformations using the receiver's domain: auto&& snd2 = transform_sender(get- domain-late(snd, get_env(rcv)), std::forward<Sender>(snd), get_env(rcv)); using Sender2 = decltype(snd2); // First, look for a customization of tag_invoke: if constexpr (tag_invocable<connect_t, Sender2, Receiver>) { return tag_invoke(*this, std::forward<Sender2>(snd2), std::forward<Receiver>(rcv)); } // Next, see if the sender is co_await-able: else if constexpr (is-await- connectable<Sender2, Receiver>) { /* ... */ } } }; </pre>

§ 2.1.2.1 Parity with coroutines

The use of `transform_sender` in `connect` it is analagous to the use of `await_transform` in `co_await`. Glossing over some details, in a coroutine the expression `co_await expr` is “lowered” to operator `co_await(p.await_transform(expr)).await_suspend(handle-to-p)`, where `p` is a reference to coroutine’s promise. This gives the coroutine task type some say in how `co_await` expressions are evaluated.

The addition of `transform_sender` to P2300 satisfies the same need to customize the launch behavior of child async tasks. An expression like `connect(sndr, detached-receiver)` is “lowered” to `connect(transform_sender(domain, sndr, get_env(detached-receiver)), detached-receiver)`, where `domain` is a property of the receiver’s environment. This gives the receiver some say in how `connect` expressions are evaluated.

§ 2.1.2.2 Recursive transformations

The author anticipates the need to sometimes apply a transformation recursively to all of a sender’s child senders. Such a generic recursive transformation might look something like this:

```

// my_domain applies a transformation recursively
auto my_domain::transform_sender(Sender&& snd, const Env& env) const {
    auto [tag, data, ...child] = snd;

    // Create a temporary sender with transformed children
    auto tmp = make_sender(
        tag,
        data,
        ex::transform_sender(*this, child, env)...);

    // Use the transformed children to compute a domain
    // (they all must share a domain or it's an error)

```

```

auto&& [x, y, ...child2] = tmp;
auto domain2 = common-domain-of(child2...);

// Use the predecessor domain to transform the temporary sender:
return ex::transform_sender(domain2, move(tmp), env);
}

```

This works well until we apply this function to a sender that modifies the environment it passes to its child operations. Take the case of `on(sch, snd)`: when it is connected to a receiver, it connects its child sender `snd` with a receiver whose environment has been modified to show that the current scheduler is `sch` (because `on` will start `snd` there).

But the implementation of `my_domain::transform_sender` above does not update the environment when it is recursively transforming an `on` sender's child. That means the child will be transformed with incorrect information about where it will be executing, which can change the meaning of the transformation. That's not good. Something is missing.

We need a way to ask a sender to apply its transformation to an environment. That is, in addition to `transform_sender` we need `transform_env` that can be used to fix the code above as follows (differences highlighted):

```

// my_domain applies a transformation recursively
auto my_domain::transform_sender(Sender&& snd, const Env& env) const {
    auto [tag, data, ...child] = snd;

    // Apply any necessary transformations to the environment
    auto&& env2 = ex::transform_env(*this, snd, env);

    // Create a temporary sender with transformed children,
    // using the transformed environment from the line above
    auto tmp = make_sender(
        tag,
        data,
        ex::transform_sender(*this, child, env2)...);

    // Use the transformed children to compute a domain
    // (they all must share a domain or it's an error)
    auto&& [x, y, ...child2] = tmp;
    auto domain2 = common-domain-of(child2...);

    // Use the predecessor domain to transform the temporary sender:
    return ex::transform_sender(domain2, move(tmp), env);
}

```

Now expressions can generically be transformed recursively.

§ 2.1.3 Early (sender construction-time) customization

We can use `transform_sender` for early customization as well as late. The benefit of doing this is that only one set of customizations needs be written for each domain, rather than two (early and late).

This paper proposes that each algorithm constructs a default sender that implements the default behavior for that algorithm. It then passes that sender to `transform_sender` along with the sender's domain. The result of `transform_sender` is what the algorithm returns.

The following comparison table shows how we propose to change the connect customization point:

Table 3: The proposed changes to the then customization point

Before	After
--------	-------

Before	After
<pre> struct then_t { template <sender Sender, class Fun> requires /* ... */ auto operator()(Sender&& snd, Fun fun) const { // First, use the completion scheduler to // look for a tag_invoke if constexpr (has-customization<then_t, set_value_t, Sender, Fun>) { auto&& env = get_env(snd); return tag_invoke(*this, get_completion_scheduler<set_value_t> (env), (snd), std::forward<Sender> (snd), std::move(fun)); } // Otherwise, use the default // implementation: else { return then_sender<Sender, Fun> (std::forward<Sender>(snd), std::move(fun)); } } }; </pre>	<pre> struct then_t { template <sender Sender, class Fun> requires /* ... */ auto operator()(Sender&& snd, Fun fun) const { // Get the domain from the // predecessor sender: auto domain = get-domain- early(snd); // Create a `then` sender // and ask the predecessor // to // transform it if desired return transform_sender(domain, then_sender<Sender, Fun> (std::forward<Sender> (snd), std::move(fun))); } }; </pre>

Some algorithms are required to do some work eagerly in their default implementation (e.g., `split`, `ensure_started`). These algorithms must first create a dummy sender to pass to `transform_sender`. The “default” domain, which is used when no other domain has been specified, can transform these dummy senders and do their eager work in the process. The same mechanism is also useful to implement customizable sender algorithms whose default implementation merely lowers to a more primitive expression (e.g. `transfer(snd, sch)` becomes `schedule_from(sch, snd)`, and `transfer_just(sch, ts...)` becomes `just(ts...) | transfer(sch)`).

For example, here is how the `transfer_just` customization point might look after the change:

Table 4: The proposed changes to the `then` customization point

Before	After
<pre> struct transfer_just_t { template <scheduler Scheduler, class... Ts> requires /* ... */ auto operator()(Scheduler sch, Ts&&... ts) const { // First, use the completion scheduler to // look for a tag_invoke if constexpr (has-customization< transfer_just_t, set_value_t, Scheduler, Ts...>) { auto&& env = get_env(snd); return tag_invoke(*this, get_completion_scheduler<set_value_t> (env), std::move(sch), std::forward<Ts> (ts)...); } } }; </pre>	<pre> struct transfer_just_t { template <scheduler Scheduler, class... Ts> requires /* ... */ auto operator()(Scheduler sch, Ts&&... ts) const { // Get the predecessor's domain auto domain = get-domain-early(snd); // Construct a transfer_just sender // and transform it return transform_sender(domain, make_sender(*this, tuple{std::move(sch), std::forward<Ts>(ts)...}); } }; </pre>

Before	After
<pre> } // Otherwise, use the default // implementation: else { return just(std::forward<Ts>(ts)...) transfer(std::move(sch)); } } }; </pre>	<pre> // default_domain::transform_sender // dispatches here: template <class Sender, class Env> auto transform_sender(Sender&& snd, const Env& env) const { auto [tag, data] = std::forward<Sender>(snd); auto [sch, ...ts] = std::move(data); return just(std::move(ts)...) transfer(std::move(sch)); } }; </pre>

Some algorithms are entirely eager with no lazy component, like `sync_wait` and `start_detached`. For these, “transforming” a sender isn’t what you want; you want to dispatch to an eager algorithm that will actually *consume* the sender. We can use domains to dispatch to the correct implementation for those as well. This paper proposes the addition of an `apply_sender` function.

The following table describes the differences between `transform_sender` and `apply_sender`:

Table 5: The differences between `transform_sender` and `apply_sender`

<code>transform_sender</code>	<code>apply_sender</code>
— always called with either a sender or a sender+env	— called with a sender and an arbitrary set of additional arguments
— always returns a sender	— can return anything
— has a sensible default implementation: identity	— has no sensible default implementation
— useful for lazy or partly lazy algorithms (e.g., <code>then</code> , <code>ensure_started</code>)	— useful for fully eager algorithms (e.g., <code>start_detached</code> , <code>sync_wait</code>)

To permit third parties to author customizable sender algorithms with partly or fully eager behavior, the mechanism by which the default domain finds the default `transform_sender` and `apply_sender` implementations shall be specified: they both dispatch to similarly named functions on the tag type of the input sender; i.e., `default_domain().transform_sender(snd, env)` is equal to `tag_of_t<decltype(snd)>().transform_sender(snd, env)`.

§ 2.1.4 Decomposable senders

For the `transform_sender` customization point to be useful, we need a way to access the constituent pieces of a sender and re-assemble it from (possibly transformed) pieces. Senders, like coroutines, generally begin in a “suspended” state; they merely curry their algorithm’s arguments into a subsequent call to connect. These “suspended” senders are colloquially known as *lazy* senders.

Each lazy sender has an associated algorithm tag, a (possibly empty) set of auxiliary data and a (possibly empty) set of child senders; e.g., the sender returned from `then(snd, fun)` has `then_t` as its tag, the set `[fun]` as its auxiliary data, and `[snd]` as its set of child senders, while `just(42, 3.14)` has `just_t` as its tag, `[42, 3.14]` as its data set and `[]` as its child set.

This paper proposes to use structured bindings as the API for decomposing a lazy sender into its tag, data, and child senders:

```
auto&& [tag, data, ...children] = snd;
```

[P1061R5], currently in Core wording review for C++26, permits the declaration of variadic structured bindings like above, making this syntax very appealing.

Not all senders are required to be decomposable, although all the “standard” lazy senders shall be. There needs to be a syntactic way to distinguish between decomposable and ordinary, non-decomposable senders (decomposable senders subsuming the sender concept).

There is currently no trait for determining whether a type can be the initializer of a structured binding. However, EWG has already approved [P2141R1] for C++26, and with it such a trait could be built, giving us a simple way to distinguish between decomposable and non-decomposable senders.

If P2141 is not adopted for C++26, we will need some other syntactic way to opt-in. One possibility is to require that the sender type’s nested `is_sender` type shall have some known, standard tag type as a base class to signify that that sender type can be decomposed.

§ 2.1.4.1 Recomposing senders

After decomposing a sender, it is often desirable to re-compose it from its modified constituents. No separate API for reconstituting senders is necessary though. It is enough to construct a decomposable sender of some arbitrary type and then pass it to `transform_sender` with an execution domain to place it in its final form.

Consider the case where `my_domain::transform_sender()` is passed `your_sender<Children...>`. It unpacks it into its tag/data/children constituents and munges the children somehow. It then wants to reconstruct a `your_sender` from the munged children. Instead, it constructs *arbitrary-sender*{tag, data, munged_children} and passes it to `execution::transform_sender()` along with `your_domain`, the domain associated with `your_sender`. That presumably will transform the *arbitrary-sender* back into a `your_sender`.

§ 2.2 Why have both early and late customization?

The full information necessary to dispatch to the correct algorithm implementation isn’t available until the execution environment is known, when we are connect-ing a sender to a receiver. Adding a connect-time search for a customization is sensible. One might wonder whether it is necessary to keep the early construction-time customization search.

With senders that can be de- and re-composed, it is true that any transformation that can be applied at sender construction time can be expressed instead as a tree transformation at connect time, so strictly speaking the early customization phase is unnecessary. There are technical and procedural reasons to keep the early customization phase, however. Here we list a few.

1. **Some transformations are more easily and efficiently applied at sender construction time.** Let’s consider a simple pipeline like the following:

```
auto snd = on( io_sched, fetch_data() )
            | transfer( cpu_sched )
            | then( compute );
```

When constructing the outermost then sender, the information about where it will execute is readily available: the left operand of the `| then(...)` expression has perfect information that the then algorithm can use immediately to pick the correct algorithm implementation.

Had we deferred the customization until connect time, we are presented with a problem: when we are connect-ing the then sender, the information about where it will execute is buried within the expression tree. connect would have to introspect the tree to compute the information, which is far more complicated.

2. **Domain-specific eager transformations can create fluid interfaces.** Users might choose to apply an eager transformation to decorate the “standard” senders in ways that make them more useful for their purposes. They might add members, adapt them to more easily interoperate with another framework, add a `co_await` operator, or add implicit conversions. They might even choose to pass each sender through `ensure_started` to get work executing before the task graph is fully built.
3. **Removing eager execution presents procedural difficulties.** Support for eager execution via customization was needed for consensus in SG1. Any attempt to remove eager execution would bounce P2300 back to SG1 where the removal is likely to be met unfavorably. And in order for that to happen, someone would have to write and champion a paper proposing the removal, and it is unclear who would do that work.

§ 2.3 An example of early and late customizations

In this section, we work through an example of a sender expression with multiple transitions between two domains. We show the interactions between a predecessor sender’s completion scheduler, and a receiver’s scheduler. In the end, we’ll see that customized senders will run on the scheduler corresponding to the domain that created them.

First, we define two domains: `domainA` and `domainB`, and we give each a customization of the `then()` algorithm that replaces the default `then` sender with a custom one that prints tracing messages to `cout`.

```
struct domainA {
    template <sender Sender, class... Env>
        requires same_as<ex::tag_of_t<Sender>, ex::then_t>
        auto transform_sender(Sender&& snd, const Env&... env) const {
            std::cout << "hello from domain A transform_sender, "
                << (sizeof...(env) ? "late" : "early") << '\n';
            auto [tag, fun, child] = (Sender&&) snd;
            return trace_then_sender(std::move(child), std::move(fun), "A");
        }
};
```

`domainB` is defined similarly. Note that this `transform_sender` member is constrained to only accept `then` senders, and that it replaces the default `then` sender with a custom one. We use the presence or absence of the second parameter, `env`, to tell whether this transformation is being done early or late and print that to `cout`.

Let’s assume the existence of an execution context that lets us schedule work onto a worker thread and that is parameterizable with an execution domain. We create two such contexts and give each one a name that gets stored in thread-local storage.

```
thread_local std::string thread_name{};

int main() {
    thread_context<domainA> ctxA("A");
    thread_context<domainB> ctxB("B");

    auto schedA = ctxA.get_scheduler();
    auto schedB = ctxB.get_scheduler();

    auto hello = [] { std::cout << "  running on thread " << thread_name << '\n'; };
    //...
}
```

Next, let’s create a sender with a transition to one of the thread execution contexts. Each line is annotated with information about the currently active domain.

```

auto work =
  ex::just()           // no domain here
| ex::then(hello)      // no domain here
| ex::transfer(schedA) // transition into domain A
| ex::then(hello);     // "hello from domain A transform_sender, early"
                      // (using predecessor's domain via completion scheduler)

```

This statement causes:

```
hello from domain A transform_sender, early
```

to be printed to cout. This is because the then that follows transfer(schedA) gets transformed by the transform_sender of domainA, the domain associated with the completion scheduler of transfer(schedA); i.e., schedA.

Next, we launch this work on thread context “B” and wait for it to complete:

```
ex::sync_wait( ex::on( schedB, work ) );
```

The composite sender passed to sync_wait has an on sender that is outermost. Stored immediately within that is the trace_then_sender, and within that the transfer, then, and just senders, with just being innermost.

sync_wait calls connect on the on sender, passing it a receiver. It is the job of on to launch its child on schedB. It tells its child where it is executing by putting schedB in the environment of the receiver that on connects it with. So, when the trace_then_sender gets connected, it sees schedB as the current scheduler in the receiver’s environment.

The connect customization point is responsible for applying any late customization by using the current execution domain to find the correct implementation. The trace_then_sender presents connect with a conundrum: the domain in the receiver is domainB, but the domain of the predecessor sender (ex::transfer(schedA)) is domainA. So which should connect use?

The answer is domainA, the domain from the predecessor sender. Looking at the expression, we can see that the outermost then will be executed on schedA, so using domainA to find the right implementation is correct. The rule is: domain information from the predecessor(s) trumps domain information from the receiver.

So we use domainA to transform the trace_then_sender. But domainA doesn’t have a transform_sender that accepts trace_then_sender; it is passed through unchanged and then connect-ed.

That in turn causes transfer(schedA) to be connected with a trace_then_receiver. There is no need to update the receiver’s scheduler; transfer runs its *continuation* on schedA. Its child is *started* on schedB so we leave the environment alone.

The transfer sender then connects its child, which is a then sender. The current domain is still domainB because the scheduler in the receiver’s environment is still schedB. The connect customization point uses domainB to transform the then sender, and the following is printed on the terminal:

```
hello from domain B transform_sender, late
```

The then sender gets transformed into a trace_then_sender, which remembers that it was created by domainB. That sender is then connected, which causes the just sender to be connected, and finally the operation state is fully constructed. The on operation state is outermost and the just operation state is innermost.

sync_wait then calls start on the on operation state, which causes execution to transition to thread “B”. There, all the nested operation states are started in turn, with the just operation state being started last.

Being the last started, it is the first to complete. That causes its parent operation to complete, its parent being the `trace_then_sender` that was created by domainB. `set_value` is invoked on `trace_then_receiver`. The `trace_then_receiver` has a `set_value` customization that looks like this:

```
void tag_invoke(ex::set_value_t, trace_then_receiver&& self) noexcept {
    std::cout << "then sender from domain " << self.name << '\n';
    self.fun();
    ex::set_value(std::move(self.rcv));
}
```

The following is printed to cout:

```
then sender from domain B
running on thread B
```

So we see that we have run “B”’s `trace_then_sender` on thread “B”, which is exactly the point.

Next to complete is the transfer to schedA, which causes execution to hop over to thread “A”. The `trace_then_sender` created by domainA completes next, causing the following to be printed:

```
then sender from domain A
running on thread A
```

So we have run “A”’s `trace_then_sender` on thread “A”. As it should be.

Finally, the on sender completes, which signals completion to the main thread and `sync_wait` returns.

In summary, the rules governing the active domain both at sender construction time and at sender connection time work together to ensure that when an algorithm is executing on a particular scheduler, it is the correct algorithm implementation that is getting executed. The dispatching is controlled by the active domain, which is used to transform senders into ones that implement the algorithm correctly for the current execution context.

The code for this example can be found in [this gist](#), which was compiled against [this revision](#) of `stdexec`, the `std::execution` reference implementation.

§ 2.4 Summary of proposed changes

In condensed form, here are the changes this paper is proposing:

1. Add a `default_domain` type for use when no other domain is determinable.
2. Add a new `get_domain(env) -> domain-tag` forwarding query.
3. A sender can publish up to 3 different completion schedulers. They can all be different, but they must all agree about the current execution domain. This paper proposes adding that as a requirement to the sender concept.
4. Add a new `transform_sender(domain, sender [, env]) -> sender` API. This function is not itself customizable, but it will be used for both early customization (at sender construction-time) and late customization (at sender/receiver connection-time).

Early customization:

- called from within each sender algorithm’s customization point object
- replaces the current mechanism of tag-dispatching to a sender factory function using the completion scheduler as a tag
- called without an environment argument

— *domain* is derived from the sender by trying the following in order:

1. `get_domain(get_env(sender))`
2. `completion-domain(sender)`, where *completion-domain* is the domain shared by all of the sender's completion signatures.
3. `default_domain()`

Late customization:

— called from the connect customization point object before tag-dispatching with `connect_t` to `tag_invoke`

— called with the receiver's environment

— *domain* is derived from the sender and the receiver by trying the following in order:

1. `get_domain(get_env(sender))`
2. `completion-domain(sender)`,
3. `get_domain(get_env(receiver))`
4. `get_domain(get_scheduler(get_env(receiver)))`
5. `default_domain()`

`transform_sender(domain, sender [, env])` returns the first of these that is well-formed:

- `domain.transform_sender(sender [, env])`
- `default_domain().transform_sender(sender [, env])`
- `sender`

If the sender returned from `transform_sender` has a different type than the one passed to it, `transform_sender` invokes itself recursively.

5. Add a `transform_env(domain, sender, env) -> env'` API in support of generic recursive sender transformations. The *domain* argument is determined from *sender* and *env* as for `transform_sender`.

`transform_env(domain, sender, env)` returns the first of these that is well-formed:

- `domain.transform_env(sender, env)`
- `default_domain().transform_env(sender, env)`

6. The standard, “lazy” sender types (i.e., those returned from sender factory and adaptor functions) return sender types that are decomposable using structured bindings into its `[tag, data, ...children]` components.
7. A call to the `when_all` algorithm should be ill-formed unless all of the sender arguments have the same domain type (as determined for senders above). The resulting `when_all` sender should publish that domain via the sender's environment.
8. The receiver that the `on(sch, snd)` algorithm uses to connect `snd` should have `sch` as the current scheduler in its environment.
9. The sender factories `just`, `just_error`, and `just_stopped` need their tag types to be specified. Name them `just_t`, `just_error_t`, and `just_stopped_t`.
10. In the algorithm `let_value(snd, fun)`, if the predecessor sender `snd` has a completion domain, then the receiver connected to the secondary sender (the one returned from `fun` when called with `snd`'s

results) shall expose that domain as the current one in the receiver's environment.

In other words, if the predecessor sender `snd` completes with values `vs...`, then the result of `fun(vs...)` will be connected to a receiver `rcv` such that `get_domain(get_env(rcv))` is equal to `completion-domain(snd)`.

The same is true also of the completion scheduler, if the predecessor has one.

So for `let_value`, likewise also for `let_error` and `let_stopped`, using `set_error_t` and `set_stopped` respectively when querying for the predecessor sender's completion scheduler and domain.

11. The `schedule_from(sch, snd)` algorithm should return a sender `snd2` such that `get_domain(get_env(snd2))` is equal to `get_domain(sch)`.

12. The following customizable algorithms, whose default implementations must do work before returning the result sender, will have their work performed in overloads of `default_domain::transform_sender`:

- `split`
- `ensure_started`

13. The following customizable algorithms, whose default implementations are trivially expressed in terms of other more primitive operations, will be lowered into their primitive forms by overloads of `default_domain::transform_sender`:

- `transfer`
- `transfer_just`
- `transfer_when_all`
- `transfer_when_all_with_variant`
- `when_all_with_variant`

14. In the algorithm `let_value(snd, fun)`, all of the sender types that the input function `fun` might return – the set of potential result senders – must all have the same domain; otherwise, the call to `let_value` is ill-formed.

Ideally, the `let_value` sender would report the result senders' domain as its domain, however we don't know the set of completions until the `let_value` sender is connected to a receiver; hence, we also don't know the set of potential result senders or their domains. Instead, we require that all the result senders share an execution domain with the predecessor sender. If they differ, connect is ill-formed.

For example, consider the following sender:

```
auto snd =
    read(get_scheduler)
    | transfer(schA)
    | let_value([], auto schB){ return schedule(schB); })
```

This reads the current scheduler from the receiver, transfers execution to `schA`, and then (indirectly, through `let_value`) transitions onto the scheduler read from the receiver (`schB`). This sender can be connect-ed only to receivers `Rcv` for which the scheduler `get_scheduler(get_env(Rcv))` has the same execution domain as that of `schA`.

Likewise for `let_error` and `let_stopped`.

This solution is not ideal. I am currently working on a more flexible solution, but I'm not yet sufficiently confident in it to propose it here.

15. Add a new `apply_sender(domain, tag, sender, args...) -> result` API. Like `transform_sender`, this function is not itself customizable, but it will be used to customize sender consuming algorithms such as `start_detached` and `sync_wait`.

- called from within each consuming sender algorithm's customization point object
- replaces the current mechanism of tag-dispatching to a custom implementation using the completion scheduler as a tag
- called the sender's execution domain, the algorithm's tag, the sender, and any additional algorithm arguments
- `domain` is determined as for `transform_sender`

`apply_sender(domain, tag, sender, args...)` returns the first of these that is well-formed:

- `domain.apply_sender(tag, sender, args...)`
- `default_domain().apply_sender(tag, sender, args...)`

16. The following customizable sender-consuming algorithms will have their default implementations in overloads of `default_domain::apply_sender`:

- `start_detached`
- `sync_wait`

§ 3 Implementation Experience

Has it been implented? YES. The design changes herein proposed are implemented in the main branch of [\[stdexecgithub\]](#), the reference implementation. The bulk of the changes including `get_domain`, `transform_sender`, and the changes to connect have been shipping since [this commit](#) on August 3, 2023 which changed the `static_thread_pool` scheduler to use `transform_sender` to parallelize the bulk algorithm.

§ 4 Proposed Wording

The following proposed changes are relative to [\[P2300R7\]](#).

[Editor's note: Change §11.4 [\[exec.syn\]](#) as follows:]

```
// [exec.queries], queries
enum class forward_progress_guarantee;
namespace queries { // exposition only
    struct get_domain_t;
    struct get_scheduler_t;
    struct get_delegatee_scheduler_t;
    struct get_forward_progress_guarantee_t;
    template<class CPO>
        struct get_completion_scheduler_t;
}

using queries::get_domain_t;
using queries::get_scheduler_t;
using queries::get_delegatee_scheduler_t;
using queries::get_forward_progress_guarantee_t;
using queries::get_completion_scheduler_t;
```



```

inline constexpr get_domain_t get_domain{};
inline constexpr get_scheduler_t get_scheduler{};
inline constexpr get_delegatee_scheduler_t get_delegatee_scheduler{};
inline constexpr get_forward_progress_guarantee_t get_forward_progress_guarantee{};
template<class CPO>
    inline constexpr get_completion_scheduler_t<CPO> get_completion_scheduler{};

// [exec.domain.default], domains
struct default_domain;

```

[Editor's note: ... and ...]

```

template<class Snd, class Env = empty_env>
    requires sender_in<Snd, Env>
    inline constexpr bool sends_stopped = see below;

template <sender Sender>
    using tag_of t = see below;

// [exec.snd.transform], sender transformations
template <class Domain, sender Sender>
    constexpr sender decltype(auto) transform_sender(Domain dom, Sender&& snd);

template <class Domain, sender Sender, queryable Env>
    constexpr sender decltype(auto) transform_sender(Domain dom, Sender&& snd, const Env&
        env);

template <class Domain, sender Sender, queryable Env>
    constexpr decltype(auto) transform_env(Domain dom, Sender&& snd, Env&& env) noexcept;

// [exec.snd.apply], sender algorithm application
template <class Domain, class Tag, sender Sender, class... Args>
    constexpr decltype(auto) apply_sender(Domain dom, Tag, Sender&& snd, Args&&... args)
        noexcept(see below);

// [exec.connect], the connect sender algorithm
namespace senders-connect { // exposition only
    struct connect_t;
}
using senders-connect::connect_t;
inline constexpr connect_t connect{};

```

[Editor's note: ... and ...]

```

// [exec.factories], sender factories
namespace senders-factories { // exposition only
    struct just_t;
    struct just_error_t;
    struct just_stopped_t;
    struct schedule_t;
    struct transfer_just_t;
}
using sender-factories::just_t;
using sender-factories::just_error_t;
using sender-factories::just_stopped_t;
inline constexpr unspecified just_t just{};
inline constexpr unspecified just_error_t just_error{};
inline constexpr unspecified just_stopped_t just_stopped{};

```

[Editor's note: After §11.5.4 [exec.get.stop.token], add the following new subsection:]

§11.5.? execution::get_domain [exec.get.domain]

1. get_domain asks an object for an associated execution domain tag.

2. The name `get_domain` denotes a query object. For some subexpression `o`, `get_domain(o)` is expression-equivalent to `mandate-nothrow-call(tag_invoke, get_domain, as_const(o))`, if this expression is well-formed.
3. `std::forwarding_query(execution::get_domain)` is true.
4. `get_domain()` (with no arguments) is expression-equivalent to `execution::read(get_domain) ([exec.read])`.

[Editor's note: To section §11.6 [exec.sched], insert a new paragraph between 6 and 7 as follows:]

7. For a given scheduler expression `sch`, if the expression `get_domain(sch)` is well-formed, then the expression `get_domain(get_env(schedule(sch)))` is also well-formed and has the same type.

[Editor's note: To section §11.9.1 [exec.snd.concepts], after paragraph 4, add two new paragraphs as follows:]

5. Let `snd` be an expression such that `decltype((snd))` is `Snd`. The type `tag_of_t<Snd>` is as follows:

- If the declaration `auto&& [tag, data, ...children] = snd;` would be well-formed, `tag_of_t<Snd>` is an alias for `decltype(auto(tag))`.
- Otherwise, `tag_of_t<Snd>` is ill-formed.

[Editor's note: There is no way in standard C++ to determine whether the above declaration is well-formed without causing a hard error, so this presumes compiler magic. However, the author anticipates the adoption of [P2141R1], which makes it possible to implement this purely in the library. P2141 has already been approved by EWG for C++26.]

6. Let *sender-for* be an exposition-only concept defined as follows:

```
template <class Sender, class Tag>
concept sender-for =
    sender<Sender> &&
    same_as<tag_of_t<Sender>, Tag>;
```

[Editor's note: After §11.9.2 [exec.awaitables], add the following new subsections:]

§11.9.? execution::default_domain [exec.domain.default]

```
struct default_domain {
    template <sender Sender>
        static constexpr sender decltype(auto) transform_sender(Sender&& snd) noexcept(see
            below);

    template <sender Sender, queryable Env>
        static constexpr sender decltype(auto) transform_sender(Sender&& snd, const Env& env)
            noexcept(see below);

    template <sender Sender, queryable Env>
        static constexpr decltype(auto) transform_env(Sender&& snd, Env&& env) noexcept;

    template <class Tag, sender Sender, class... Args>
        static constexpr decltype(auto) apply_sender(Tag, Sender&& snd, Args&&... args)
            noexcept(see below);
};
```

§11.9.?1 Static members [exec.domain.default.statics]

```
template <sender Sender>
    constexpr sender decltype(auto) default_domain::transform_sender(Sender&& snd)
        noexcept(see below);
```

Returns: `tag_of_t<Sender>().transform_sender(std::forward<Sender>(snd))` if that expression is well-formed; otherwise, `std::forward<Sender>(snd)`.

Remarks: The exception specification is equivalent to:

```
noexcept(tag_of_t<Sender>().transform_sender(std::forward<Sender>(snd)))
```

if that expression is well-formed; otherwise, `true`;

```
template <sender Sender, queryable Env>
constexpr sender decltype(auto) default_domain::transform_sender(Sender&& snd, const
    Env& env) noexcept(see below);
```

Returns: `tag_of_t<Sender>().transform_sender(std::forward<Sender>(snd), env)` if that expression is well-formed; otherwise, `std::forward<Sender>(snd)`.

Remarks: The exception specification is equivalent to:

```
noexcept(tag_of_t<Sender>().transform_sender(std::forward<Sender>(snd), env))
```

if that expression is well-formed; otherwise, `true`;

```
template <sender Sender, queryable Env>
constexpr decltype(auto) default_domain::transform_env(Sender&& snd, Env&& env)
    noexcept;
```

Returns: `tag_of_t<Sender>().transform_env(std::forward<Sender>(snd), std::forward<Env>(env))` if that expression is well-formed; otherwise, `static_cast<Env>(std::forward<Env>(env))`.

Mandates: The selected expression in *Returns:* is not potentially throwing.

```
template <class Tag, sender Sender, class... Args>
static constexpr decltype(auto) default_domain::apply_sender(Tag, Sender&& snd,
    Args&&... args) noexcept(see below);
```

Returns: `Tag().apply_sender(std::forward<Sender>(snd), std::forward<Args>(args)...) if that expression is well-formed; otherwise, this function shall not participate in overload resolution.`

Remarks: The exception specification is equivalent to:

```
noexcept(Tag().apply_sender(std::forward<Sender>(snd), std::forward<Args>
    (args)...))
```

§11.9.? execution::transform_sender [exec.snd.transform]

```
template <class Domain, sender Sender>
constexpr sender decltype(auto) transform_sender(Domain dom, Sender&& snd);

template <class Domain, sender Sender, class Env>
constexpr sender decltype(auto) transform_sender(Domain dom, Sender&& snd, const Env&
    env);
```

Returns: Let *ENV* be a parameter pack consisting of the single expression *env* for the second overload and an empty pack for the first. Let *snd2* be the expression `dom.transform_sender(std::forward<Sender>(snd), ENV...)` if that expression is well-formed; otherwise, `default_domain().transform_sender(std::forward<Sender>(snd), ENV...)`. If *snd2* and *snd* have the same type ignoring cv qualifiers, returns *snd2*; otherwise, `transform_sender(dom, snd2, ENV...)`.

```
template <class Domain, sender Sender, queryable Env>
constexpr decltype(auto) transform_env(Domain dom, Sender&& snd, Env&& env) noexcept;
```

Returns: `dom.transform_sender(std::forward<Sender>(snd), std::forward<Env>(env))` if that expression is well-formed; otherwise, `default_domain().transform_sender(std::forward<Sender>(snd), std::forward<Env>(env))`.

§11.9.? `execution::apply_sender` [`exec.snd.apply`]

```
template <class Domain, class Tag, sender Sender, class... Args>
constexpr decltype(auto) apply_sender(Domain dom, Tag, Sender&& snd, Args&&... args)
    noexcept(see below);
```

Returns: `dom.apply_sender(Tag(), std::forward<Sender>(snd), std::forward<Args>(args)...) if that expression is well-formed; otherwise, default_domain().apply_sender(Tag(), std::forward<Sender>(snd), std::forward<Args>(args)...) if that expression is well-formed; otherwise, this function shall not participate in overload resolution.`

Remarks: The exception specification is equivalent to:

```
noexcept(dom.apply_sender(Tag(), std::forward<Sender>(snd), std::forward<Args>(args)...))
```

if that expression is well-formed; otherwise,

```
noexcept(default_domain().apply_sender(Tag(), std::forward<Sender>(snd),
std::forward<Args>(args)...))
```

[Editor's note: Add a paragraph to §11.9 [`exec.snd`]]

1. This section makes use of the following exposition-only entities.

```
1. template <class Default = default_domain, class Sender>
constexpr auto completion_domain(const Sender& snd) noexcept;
```

Effects: Let `COMPL-DOMAIN(T)` be the type of the expression `get_domain(get_completion_scheduler<T>(get_env(snd)))`. If `COMPL-DOMAIN(set_value_t, snd)`, `COMPL-DOMAIN(set_error_t, snd)`, and `COMPL-DOMAIN(set_stopped_t, snd)` all share a common type [meta.trans.other] (ignoring those types that are ill-formed), then `completion_domain<Default>(snd)` is a default-constructed prvalue of that type. Otherwise, if all of those types are ill-formed, `completion_domain<Default>(snd)` is a default-constructed prvalue of type `Default`. Otherwise, `completion_domain<Default>(snd)` is ill-formed.

```
2. template <class Tag, class Env, class Default>
constexpr decltype(auto) query-with-default(Tag, const Env& env, Default&&
value) noexcept(see below);
```

Effects: Equivalent to:

— return `Tag()(env)`; if that expression is well-formed,

— return `static_cast<Default>(std::forward<Default>(value))`; otherwise.

Remarks: The expression in the `noexcept` clause is:

```
is_invocable_v<Tag, const Env&> ? is_nothrow_invocable_v<Tag, const
Env&>
                                : is_nothrow_constructible_v<Default,
Default>
```

```
3. template <class Sender>
constexpr auto get-domain-early(const Sender& snd) noexcept;
```

Effects: Equivalent to the first of the following that is well-formed:

- return `get_domain(get_env(snd))`;
- return `completion-domain(snd)`;
- return `default_domain()`;

```
4. template <class Sender, class Env>
  constexpr auto get-domain-late(const Sender& snd, const Env& env) noexcept;
```

Effects: Equivalent to:

- If `sender-for<Sender, transfer_t>` is true, then return `query-or-default(get_domain, sch, default_domain())` where `sch` is the scheduler that was used to construct `snd`,
- Otherwise, return `get_domain(get_env(snd))`; if that expression is well-formed,
- Otherwise, return `completion-domain<X>(snd)`; if that expression is well-formed and its type is not `X` where `X` is an unspecified type,
- Otherwise, return `get_domain(env)`; if that expression is well-formed,
- Otherwise, return `get_domain(get_scheduler(env))`; if that expression is well-formed,
- Otherwise, return `default_domain()`;

[*Note:* The transfer algorithm is unique in that it ignores the execution domain of its predecessor, using only its destination scheduler to select a customization. — *end note*]

```
5. template <class... T>
  struct product-type {
    T0 t0;      // exposition only
    T1 t1;      // exposition only
    ...
    Tn-1 tn-1;  // exposition only
  };
```

— [*Note:* An expression of type *product-type* is usable as the initializer of a structured binding declaration [dcl.struct.bind]. — *end note*]

```
6. template <semiregular Tag, movable-value Data = see below, sender... Child>
  constexpr auto make-sender(Tag, Data&& data, Child&&... child);
```

Remarks: The default template argument for the `Data` template parameter names and unspecified empty trivial class type.

Returns: A prvalue of type `basic-sender<Tag, decay_t<Data>, decay_t<Child>...>` where the `tag` member has been default-initialized and the `data` and `childn...` members have been direct initialized from their respective forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below:

```
template <class Tag, class Data, class... Child> // arguments are not
  associated entities ([lib.tmpl-heads])
struct basic-sender : unspecified {
  using is_sender = unspecified;

  [[no_unique_address]] Tag tag; // exposition only
  Data data;                    // exposition only
```

```

Child0 child0;    // exposition only
Child1 child1;    // exposition only
...
Childn-1 childn-1; // exposition only
};

```

— It is unspecified whether instances of *basic-sender* can be aggregate initialized.

— The unspecified base type has no non-static data members. It may define member functions or hidden friend functions ([hidden.friends]).

— [*Note:* An expression of type *basic-sender* is usable as the initializer of a structured binding declaration [dcl.struct.bind]. — *end note*]

[Editor's note: To §11.9.5.2 [exec.just], add a paragraph 6 as follows:]

6. When used as the initializer of a structured binding declaration, expressions of type *just-sender*<Tag, Ts...> behave as do expressions of type *basic-sender*<Tag, *product-type*<Ts...>>.

[Editor's note: Change §11.9.5.3 [exec.transfer.just] as follows (some identifiers in this section have had their names changed for the sake of clarity; the name changes have not been marked up):]

2. The name `transfer_just` denotes a customization point object. For some subexpression `sch` and pack of subexpressions `vs`, let `Sch` be `decltype((sch))` and let `Vs` be the template parameter pack `decltype((vs))...`. If `Sch` does not satisfy `scheduler`, or any type `V` in `Vs` does not satisfy *movable-value*, `transfer_just(sch, vs...)` is ill-formed. Otherwise, `transfer_just(sch, vs...)` is expression-equivalent to:

```

transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_just, product-type{sch, vs...}));

```

1. ~~tag_invoke(transfer_just, sch, vs...), if that expression is valid.~~

3. Let `as` be a pack of rvalue subexpressions of types `decay_t<Vs>...` referring to objects direct-initialized from `vs`. If ~~the function selected by tag_invoke~~ a sender `out_snd` returned from `transfer_just(sch, vs...)` is connected with a receiver `rcv` with environment `env` such that `transform_sender(get-domain-late(out_snd, env), out_snd, Env)` does not return a sender whose asynchronous operations execute value completion operations on an execution agent belonging to the execution resource associated with `sch`, with value result datums `as`, the behavior of calling ~~`transfer_just(sch, vs...)`~~ `connect(out_snd, rcv)` is undefined.

— *Mandates:* `sender_of<RSnd, set_value_t(decay_t<Vs>...), Env>`, where ~~`RSnd`~~ `RSnd` is the type of ~~the tag_invoke-expression above~~ `transfer_just(sch, vs...)`, and `Env` is the type of an environment.

2. Otherwise, `transfer(just(vs...), sch)`.

4. For some subexpression `sch` and pack of subexpressions `vs`, let `out_snd` be a subexpression referring to an object returned from `transform_just(sch, vs...)` or a copy of such. Then `get_completion_scheduler<set_value_t>(get_env(out_snd)) == sch` is true, `get_completion_scheduler<set_stopped_t>(get_env(out_snd)) == sch` is true, and `get_domain(get_env(out_snd))` is expression-equivalent to `get_domain(sch)`.

5. Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))`. If *sender-for*<`Snd`, `transfer_just_t`> is false, then the expression `transfer_just_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```

auto [tag, data] = snd;
auto& [sch, ...vs] = data;
return transfer(just(std::move(vs)...), std::move(sch));

```

[*Note:* This causes the `transform_just(sch, vs...)` sender to become `transform(just(vs...), sch)` when it is connected with a receiver whose execution domain does not customize `transform_just`. — *end note*]

[Editor's note: Update §11.9.6.3 [exec.on] as follows:]

1. `on` adapts an input sender into a sender that will start on an execution agent belonging to a particular scheduler's associated execution resource.
2. Let `replace-scheduler(env, sch)` be an expression denoting an object `env2` such that `get_scheduler(env2)` returns a copy of `sch`, `get_domain(env2)` is expression-equivalent to `get_domain(sch)`, and `tag_invoke(tag, env2, args...)` is expression-equivalent to `tag(env, args...)` for all arguments `args...` and for all `tag` whose type satisfies *forwarding-query* and is not `get_scheduler_t` or `get_domain_t`.
3. The name `on` denotes a customization point object. For some subexpressions `sch` and `snd`, let `Sch` be `decltype((sch))` and `Snd` be `decltype((snd))`. If `Sch` does not satisfy *scheduler*, or `Snd` does not satisfy *sender*, `on(sch, snd)` is ill-formed. Otherwise, the expression `on(sch, snd)` is expression-equivalent to:

```

transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(on, sch, snd));

```

1. ~~`tag_invoke(on, sch, snd)`, if that expression is valid. If the function selected above~~ If a sender `out_snd` returned from `on(sch, snd)` is connected with a receiver `rcv` with environment `env` such that `transform_sender(get-domain-late(out_snd, env), out_snd, env)` does not return a sender that starts `snd` on an execution agent of the associated execution resource of `sch` when started, the behavior of calling ~~`on(sch, snd)`~~ `connect(out_snd, rcv)` is undefined.
- ~~*Mandates:* The type of the `tag_invoke` expression above satisfies *sender*.~~
4. Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))`. If `sender-for<Snd, on t>` is false, then the expression `on t().transform_sender(snd, env)` is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender `snd1` ~~. When~~ such that when `snd1` is connected with some receiver `out_rcv`, it:

1. Constructs a receiver `rcv` such that:
 1. When `set_value(rcv)` is called, it calls `connect(snd, rcv2)`, where `rcv2` is as specified below, which results in `op_state3`. It calls `start(op_state3)`. If any of these throws an exception, it calls `set_error` on `out_rcv`, passing `current_exception()` as the second argument.
 2. `set_error(rcv, err)` is expression-equivalent to `set_error(out_rcv, err)`.
 3. `set_stopped(rcv)` is expression-equivalent to `set_stopped(out_rcv)`.
 4. `get_env(rcv)` is expression-equivalent to `get_env(out_rcv)`.
2. Calls `schedule(sch)`, which results in `snd2`. It then calls `connect(snd2, rcv)`, resulting in `op_state2`.
3. `op_state2` is wrapped by a new operation state, `op_state1`, that is returned to the caller.

4. rcv2 is a receiver that wraps a reference to out_rcv and forwards all completion operations to it. In addition, get_env(rcv2) returns *replace-scheduler*(env, sch).
5. When start is called on op_state1, it calls start on op_state2.
6. The lifetime of op_state2, once constructed, lasts until either op_state3 is constructed or op_state1 is destroyed, whichever comes first. The lifetime of op_state3, once constructed, lasts until op_state1 is destroyed.

5. Let snd and env be subexpressions such that Snd is decltype((snd)). If sender-for<Snd, on t> is false, then the expression on t().transform env(snd, env) is ill-formed; otherwise, let sch be the scheduler used to construct snd. on t().transform env(snd, env) is equal to *replace-scheduler*(env, sch).

6. Given subexpressions snd1 and env, where snd1 is a sender returned from on or a copy of such, let Snd1 be decltype((snd1)). Let Env2 be decltype(*replace-scheduler*(env, sch)). Then the type of tag_invoke(get_completion_signatures, snd1, env) shall be:

```
make_completion_signatures<
  copy_cvref_t<Snd1, Snd>,
  Env2,
  make_completion_signatures<
    schedule_result_t<Sch>,
    Env,
    completion_signatures<set_error_t(exception_ptr)>,
    no-value-completions>>;
```

where *no-value-completions*<As...> names the type completion_signatures<> for any set of types As...

[Editor's note: Update §11.9.6.4 [exec.transfer] as follows:]

1. transfer adapts a sender into a sender with a different associated set_value completion scheduler. [Note: It results in a transition between different execution resources when executed. — end note]
2. The name transfer denotes a customization point object. For some subexpressions sch and snd, let Sch be decltype((sch)) and Snd be decltype((snd)). If Sch does not satisfy scheduler, or Snd does not satisfy sender, transfer(snd, sch) is ill-formed. Otherwise, the expression transfer(snd, sch) is expression-equivalent to:

```
transform_sender(
  get-domain-early(snd),
  make-sender(transfer, sch, snd));
```

1. tag_invoke(transfer, get_completion_scheduler<set_value_t>(get_env(snd)), snd, sch), if that expression is valid.

— Mandates: The type of the tag_invoke expression above satisfies sender.

2. Otherwise, tag_invoke(transfer, snd, sch), if that expression is valid.

— Mandates: The type of the tag_invoke expression above satisfies sender.

3. Otherwise, schedule_from(sch, snd).

~~If the function selected above~~ If a sender out_snd returned from transfer(snd, sch) is connected with a receiver rcv with environment env such that transform_sender(get-domain-late(out_snd, env), out_snd, env) does not return a sender ~~which~~ that is a result of a call to transform_sender(get-domain-late(out_snd, env), schedule_from(sch, snd2), env),

where `snd2` is a sender ~~which~~that sends values ~~equivalent~~equal to those sent by `snd`, the behavior of calling ~~`transfer(snd, sch)`~~ `connect(out_snd, rcv)` is undefined.

- For a sender `out_snd` returned from `transfer(snd, sch)`, `get_env(out_snd)` shall return a queryable object `q` such that `get_domain(q)` is expression-equivalent to `get_domain(sch)` and `get_completion_scheduler<CPO>(q)` returns a copy of `sch`, where `CPO` is either `set_value_t` or `set_stopped_t`. The `get_completion_scheduler<set_error_t>` query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object. For all other query objects `Q` whose type satisfies *forwarding-query*, the expression `Q(q, args...)` shall be equivalent to `Q(get_env(snd), args...)`.
- Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))`. If *sender-for*`<Snd, transfer_t>` is false, then the expression `transfer_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```
auto [tag, data, child] = snd;
return schedule_from(std::move(data), std::move(child));
```

[*Note*: This causes the `transfer(snd, sch)` sender to become `schedule_from(sch, snd)` when it is connected with a receiver whose execution domain does not customize transfer. — *end note*]

[Editor's note: Update §11.9.6.5 [exec.schedule.from] as follows:]

- `schedule_from` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource. [*Note*: `schedule_from` is not meant to be used in user code; it is used in the implementation of `transfer`. — *end note*]
- The name `schedule_from` denotes a customization point object. For some subexpressions `sch` and `snd`, let `Sch` be `decltype((sch))` and `Snd` be `decltype((snd))`. If `Sch` does not satisfy `scheduler`, or `Snd` does not satisfy `sender`, `schedule_from(sch, snd)` is ill-formed. Otherwise, the expression `schedule_from(sch, snd)` is expression-equivalent to:

```
transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make-schedule-from-sender(sch, snd));
```

where *make-schedule-from-sender*(`sch, snd`) is expression-equivalent to *make-sender*(`schedule_from, sch, snd`) and returns a sender object `snd2` that behaves as follows:

- `tag_invoke(schedule_from, sch, snd)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that completes on an execution agent belonging to the associated execution resource of `sch` and completing with the same `async_ops` as `snd`, the behavior of calling `schedule_from(sch, snd)` is undefined.

— *Mandates*: The type of the `tag_invoke` expression above satisfies `sender`.

- ~~Otherwise, constructs a sender `snd2`~~: When `snd2` is connected with some receiver `out_rcv`, it:
 - Constructs a receiver `rcv` such that when a receiver completion operation `Tag(rcv, args...)` is called, it decay-copies `args...` into `op_state` (see below) as ~~`args`~~`args2`... and constructs a receiver `rcv2` such that:
 - When `set_value(rcv2)` is called, it calls `Tag(out_rcv, std::move(argsargs2))`.
 - `set_error(rcv2, err)` is expression-equivalent to `set_error(out_rcv, err)`.
 - `set_stopped(rcv2)` is expression-equivalent to `set_stopped(out_rcv)`.

4. get_env(rcv2) is equal to get_env(rcv).

It then calls `schedule(sch)`, resulting in a sender `snd3`. It then calls `connect(snd3, rcv2)`, resulting in an operation state `op_state3`. It then calls `start(op_state3)`. If any of these throws an exception, it catches it and calls `set_error(out_rcv, current_exception())`. If any of these expressions would be ill-formed, then `Tag(rcv, args...)` is ill-formed.

2. Calls `connect(snd, rcv)` resulting in an operation state `op_state2`. If this expression would be ill-formed, `connect(snd2, out_rcv)` is ill-formed.
3. Returns an operation state `op_state` that contains `op_state2`. When `start(op_state)` is called, calls `start(op_state2)`. The lifetime of `op_state3` ends when `op_state` is destroyed.

2. [Editor's note: This para is taken from the removed para (1) above.] ~~If the function selected by tag_invoke~~If a sender out_snd returned from schedule_from(sch, snd) is connected with a receiver rcv with environment env such that transform_sender(get_domain_late(out_snd, env), out_snd, env). does not return a sender that completes on an execution agent belonging to the associated execution resource of `sch` and ~~completing~~ with the same async result (`[async.ops]`) as `snd`, the behavior of calling ~~`schedule_from(sch, snd)`~~ `connect(out_snd, rcv)` is undefined.

3. Given subexpressions `snd2` and `env`, where `snd2` is a sender returned from `schedule_from` or a copy of such, let `Snd2` be `decltype((snd2))` and let `Env` be `decltype((env))`. Then the type of `tag_invoke(get_completion_signatures, snd2, env)` shall be:

```
make_completion_signatures<
    copy_cvref_t<Snd2, Snd>,
    Env,
    make_completion_signatures<
        schedule_result_t<Sch>,
        Env,
        potentially-throwing-completions,
        no-completions>,
        value-completions,
        error-completions>;
```

where *potentially-throwing-completions*, *no-completions*, *value-completions*, and *error-completions* are defined as follows:

```
template <class... Ts>
using all-nothrow-decay-copyable =
    boolean_constant<(is_nothrow_constructible_v<decay_t<Ts>, Ts> &&
        ...)>

template <class... Ts>
using conjunction = boolean_constant<(Ts::value &&...)>;

using potentially-throwing-completions =
    conditional_t<
        error_types_of_t<copy_cvref_t<Snd2, Snd>, Env, all-nothrow-decay-
            copyable>::value &&
            value_types_of_t<copy_cvref_t<Snd2, Snd>, Env, all-nothrow-
                decay-copyable, conjunction>::value,
        completion_signatures<>,
        completion_signatures<set_error_t(exception_ptr)>;

template <class...>
using no-completions = completion_signatures<>;
```

```

template <class... Ts>
using                               value-completions                               =
    completion_signatures<set_value_t(decay_t<Ts>&&...)>;

template <class T>
using                               error-completions                               =
    completion_signatures<set_error_t(decay_t<T>&&)>;

```

3. For a sender `out_snd` returned from `schedule_from(sch, snd)`, `get_env(out_snd)` shall return a queryable object `q` such that `get_domain(q)` is expression-equivalent to `get_domain(sch)` and `get_completion_scheduler<CPO>(q)` returns a copy of `sch`, where `CPO` is either `set_value_t` or `set_stopped_t`. The `get_completion_scheduler<set_error_t>` query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object. For all other query objects `Q` whose type satisfies *forwarding-query*, the expression `Q(q, args...)` shall be equivalent to `Q(get_env(snd), args...)`.

[Editor's note: Update §11.9.6.6 [exec.then] (with analogous changes to §11.9.6.7 [exec.upon.error] and §11.9.6.8 [exec.upon.stopped]) as follows:]

1. then attaches an invocable as a continuation for an input sender's value completion operation.
2. The name then denotes a customization point object. For some subexpressions `snd` and `f`, let `Snd` be `decltype((snd))`, let `F` be the decayed type of `f`, and let ~~`f1`~~`f2` be an xvalue referring to an object decay-copied from `f`. If `Snd` does not satisfy sender, or `F` does not model *movable-value*, then `(snd, f)` is ill-formed. Otherwise, the expression `then(snd, f)` is expression-equivalent to:

```

transform_sender(
    get-domain-early(snd),
    make-then-sender(f, snd));

```

where `make-then-sender(f, snd)` is expression-equivalent to `make-sender(then, f, snd)` and returns a sender object `snd2` that behaves as follows:

1. `tag_invoke(then, get_completion_scheduler<set_value_t>(get_env(snd)), snd, f)`, if that expression is valid.
 - *Mandates:* The type of the `tag_invoke` expression above satisfies sender.
2. Otherwise, `tag_invoke(then, snd, f)`, if that expression is valid.
 - *Mandates:* The type of the `tag_invoke` expression above satisfies sender.
1. ~~Otherwise, constructs a sender `snd2`:~~ When `snd2` is connected with some receiver `out_rcv`, it:
 1. Constructs a receiver `rcv` such that:
 1. When `set_value(rcv, args...)` is called, let `v` be the expression `invoke(f1f2, args...)`. If `decltype(v)` is void, evaluates the expression `(v, set_value(out_rcv))`; otherwise, `set_value(out_rcv, v)`. If any of these throw an exception, it catches it and calls `set_error(out_rcv, current_exception())`. If any of these expressions would be ill-formed, the expression `set_value(rcv, args...)` is ill-formed.
 2. `set_error(rcv, err)` is expression-equivalent to `set_error(out_rcv, err)`.
 3. `set_stopped(rcv)` is expression-equivalent to `set_stopped(out_rcv)`.
 2. Returns an expression-equivalent to `connect(snd, rcv)`.

3. ~~Let *compl-sig-t*<Tag, Args...> name the type Tag() if Args... is a template parameter pack containing the single type void; otherwise, Tag(Args...).~~ Given subexpressions *out_snd* and *env* where *out_snd* is a sender returned from *then* or a copy of such, let *OutSnd* be *decltype*((*out_snd*)) and let *Env* be *decltype*((*env*)). The type of *tag_invoke*(*get_completion_signatures*, *out_snd*, *env*) shall be **equivalent to**:

```
make_completion_signatures<>
  copy_cvref_t<OutSnd, Snd>, Env, set-error-signature,
  set-value-completions> ÷
```

where *set-value-completions* is **an alias for the alias template**:

```
template<class... As>
  set-value-completions =
    completion_signatures<compl-sig-tset_value_t, SET-VALUE-
      SIG(invoke_result t<F, As...>)>>
```

and *set-error-signature* is an alias for *completion_signatures*<*set_error_t*(*exception_ptr*)> if any of the types in the *type-list* named by *value_types_of_t*<*copy_cvref_t*<*OutSnd*, *Snd*>, *Env*, *potentially-throwing*, *type-list*> are *true_type*; otherwise, *completion_signatures*<>, where *potentially-throwing* is the **template alias alias template**:

```
template<class... As>
  using potentially-throwing =
    bool_constant!is_nothrow_invocable_v<F, As...>>
    negation<is_nothrow_invocable<F, As...>>;
```

2. ~~If the function selected above~~ Let *out_snd* be the result of calling *then*(*snd*, *f*) or a copy of such. If *out_snd* is connected with a receiver *rcv* with environment *env* such that *transform_sender*(*get_domain_late*(*out_snd*, *env*), *out_snd*, *env*) does not return a sender that; [Editor's note: reformatted as a list for comprehensibility:]

— invokes *f* with the value result datums of *snd*,

— using *f*'s return value as ~~the sender's~~ *out_snd*'s value completion, and

— forwards the non-value completion operations to *rcv* unchanged,

then the behavior of calling ~~*then*(*snd*, *f*)~~*connect*(*out_snd*, *rcv*) is undefined.

[Editor's note: Change §11.9.6.9 [exec.let] as follows:]

1. *let_value* transforms a sender's value completion into a new child asynchronous operation.
let_error transforms a sender's error completion into a new child asynchronous operation.
let_stopped transforms a sender's stopped completion into a new child asynchronous operation.

2. [Editor's note: Copied from below:] Let the expression *let-cpo* be one of *let_value*, *let_error*, or *let_stopped* and let *set-cpo* be the completion function that corresponds to *let-cpo* (*set_value* for *let_value*, etc.). For subexpressions *snd* and *re*, let *inner-env*(*snd*, *re*) be an environment *env* such that:

— *get_domain*(*env*) is expression-equivalent *get_domain_late*(*snd*, *re*)

— *get_scheduler*(*env*) is expression-equivalent to the first well-formed expression below:

— *get_completion_scheduler*<*set-cpo-t*>(get_env(*snd*)), where *set-cpo-t* is the type of *set-cpo*.

— `get_scheduler(re)`

or if neither of them are, `get_scheduler(env)` is ill-formed.

— For all other query objects Q and arguments $\text{args}...$, $Q(\text{env}, \text{args}...)$ is expression-equivalent to $Q(\text{re}, \text{args}...)$.

3. The names `let_value`, `let_error`, and `let_stopped` denote customization point objects. ~~Let the expression `let-cpo` be one of `let_value`, `let_error`, or `let_stopped`.~~ For subexpressions snd and f , let Snd be `decltype((snd))`, let F be the decayed type of f , and let f2 be an xvalue that refers to an object decay-copied from f . If Snd does not satisfy `sender`, the expression `let-cpo(snd, f)` is ill-formed. If F does not satisfy `invocable`, the expression `let_stopped(snd, f)` is ill-formed. Otherwise, the expression `let-cpo(snd, f)` is expression-equivalent to:

```
transform_sender(  
    get-domain-early(snd),  
    make-let-sender(f, snd));
```

where `make-let-sender(f, snd)` is expression-equivalent to `make-sender(let-cpo, f, snd)` and returns a sender object snd2 that behaves as follows:

1. When snd2 is connected to some receiver rcv , it:

1. ~~`tag_invoke(let-cpo, get_completion_scheduler<set_value_t>(get_env(snd)), snd, f)`~~, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, ~~`tag_invoke(let-cpo, snd, f)`~~, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

3. Otherwise, given a receiver out_rcv and an lvalue $\text{out_rcv}'$ referring to an object decay-copied from out_rcv .

1. For `let_value`, let `set-cpo` be `set_value`. For `let_error`, let `set-cpo` be `set_error`. For `let_stopped`, let `set-cpo` be `set_stopped`. Let *completion-function* be one of `set_value`, `set_error`, or `set_stopped`.

1. Decay-copies out_rcv into op_state2 (see below). out_rcv2 is an xvalue referring to the copy of out_rcv .

2. ~~Let rcv be an rvalue of a receiver type Rcv .~~ Constructs a receiver rcv such that such that:

1. When `set-cpo(rcv, args...)` is called, the receiver rcv decay-copies $\text{args}...$ into op_state2 as $\text{args2}...$, then calls `invoke(f2, args2...)` resulting in a sender snd3 . It then calls `connect(snd3, std::move(out_rcv')out_rcv3)`, resulting in an operation state op_state3 , where out_rcv3 is a receiver described below. op_state3 is saved as a part of op_state2 . It then calls `start(op\_state3)`. If any of these throws an exception, it catches it and calls `set_error(std::move(out_rcv')out_rcv2, current_exception())`. If any of these expressions would be ill-formed, `set-cpo(rcv, args...)` is ill-formed.

2. ~~*completion-function*~~*CF*(rcv , $\text{args}...$) is expression-equivalent to ~~`completion-function(std::move(out_rcv'), args...)`~~ when

~~completion-function is different from set-cpo~~ CF(out_rcv2, __args...), where CF is a completion function other than set-cpo.

3. get_env(rcv) is expression-equivalent to get_env(out_rcv).
 4. out_rcv3 is a receiver that forwards its completion operations to out_rcv2 and for which get_env(out_rcv3) returns inner-env(get_env(snd), get_env(out_rcv2)).
 3. ~~let-cpo(snd, f) returns a sender snd2 such that~~ Calls connect(snd, rcv) resulting in an operation state op_state2. [Editor's note: The formatting is changed here.] If the expression connect(snd, rcv) is ill-formed, connect(snd2, out_rcv) is ill-formed.
 4. ~~Otherwise, let op_state2 be the result of connect(snd, rcv): connect(snd2, out_rcv) returns~~ Returns an operation state op_state that stores op_state2. start(op_state) is expression-equivalent to start(op_state2).
4. Given subexpressions out_snd and env, where out_snd is a sender returned from let-cpo(snd, f) or a copy of such, let OutSnd be decltype((out_snd)), let Env be decltype((env)), and let DS be copy_cvref_t<OutSnd, Snd>. Then the type of tag_invoke(get_completion_signatures, out_snd, env) is specified as follows:
1. If sender_in<DS, Env> is false, the expression tag_invoke(get_completion_signatures, out_snd, env) is ill-formed.
 2. Otherwise, let Sigs... be the set of template arguments of the completion_signatures specialization named by completion_signatures_of_t<DS, Env>, let Sigs2... be the set of function types in Sigs... whose return type is set-cpo, and let Rest... be the set of function types in Sigs... but not Sigs2....
 3. For each Sig2_i in Sigs2..., let Vs_i... be the set of function arguments in Sig2_i and let Snd3_i be invoke_result_t<F, decay_t<Vs_i&...>. If Snd3_i is ill-formed, or if get-domain-early(declval<Snd3_i>()) has a different type than get-domain-early(snd)., or if sender_in<Snd3_i, EnvEnv2> is not satisfied where Env2 is the type of inner-env(get_env(snd), env)., then the expression tag_invoke(get_completion_signatures, out_snd, env) is ill-formed.
 4. Otherwise, let Sigs3_i... be the set of template arguments of the completion_signatures specialization named by completion_signatures_of_t<Snd3_i, EnvEnv2>. Then the type of tag_invoke(get_completion_signatures, out_snd, env) shall be equivalent to completion_signatures<Sigs3₀..., Sigs3₁..., ... Sigs3_{n-1}..., Rest..., set_error_t(exception_ptr)>, where n is sizeof...(Sigs2).
5. Let snd and env be subexpressions such that Snd is decltype((snd)) and Env is decltype((env)). If sender-for<Snd, let-cpo-t> is false where let-cpo-t is the type of let-cpo, then the expression let-cpo-t().transform_env(snd, env) is ill-formed. Otherwise, it is equal to inner-env(get_env(snd), env).
6. If a sender out_snd returned from let-cpo(snd, f) is connected to a receiver rcv with environment env such that transform_sender(get-domain-late(out_snd, env), out_snd, env) does not return a sender that [Editor's note: reformatted as a list for comprehensibility]:
- invokes f when set-cpo is called with snd's result datums, ~~and~~
 - makes its completion dependent on the completion of a sender returned by f, and

— propagates the other completion operations sent by `snd`,

the behavior of calling `let-cpo(snd, f)connect(out_snd, rcv)` is undefined.

[Editor's note: Change §11.9.6.10 [exec.bulk] as follows:]

1. `bulk` runs a task repeatedly for every index in an index space.
2. The name `bulk` denotes a customization point object. For some subexpressions `snd`, `shape`, and `f`, let `Snd` be `decltype((snd))`, `Shape` be `decltype((shape))`, and `F` be `decltype((f))`. If `Snd` does not satisfy sender or `Shape` does not satisfy integral, `bulk` is ill-formed. Otherwise, the expression `bulk(snd, shape, f)` is expression-equivalent to:

```
transform_sender(  
    get-domain-early(snd),  
    make-bulk-sender(product-type{shape, f}, snd));
```

where `make-bulk-sender(t, snd)` is expression-equivalent to `make-sender(bulk, t, snd)` for a subexpression `t` and returns a sender object `snd2` that behaves as follows:

1. `tag_invoke(bulk, get_completion_scheduler<set_value_t>(get_env(snd)), snd, shape, f)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(bulk, snd, shape, f)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies sender.

1. ~~Otherwise, constructs a sender `snd2`:~~ When `snd2` is connected with some receiver `out_rcv`, it:

1. Constructs a receiver `rcv`:

1. When `set_value(rcv, args...)` is called, calls `f(i, args...)` for each `i` of type `Shape` from `0` to `shape`, then calls `set_value(out_rcv, args...)`. If any of these throws an exception, it catches it and calls `set_error(out_rcv, current_exception())`. If any of these expressions are ill-formed, `set_value(rcv, args...)` is ill-formed.

2. When `set_error(rcv, err)` is called, calls `set_error(out_rcv, err)`.

3. When `set_stopped(rcv)` is called, calls `set_stopped(out_rcv, env)`.

2. Calls `connect(snd, rcv)`, which results in an operation state `op_state2`.

3. Returns an operation state `op_state` that contains `op_state2`. When `start(op_state)` is called, calls `start(op_state2)`.

4. Given subexpressions `snd2` and `env` where `snd2` is a sender returned from `bulk` or a copy of such, let `Snd2` be `decltype((snd2))`, let `Env` be `decltype((env))`, let `DS` be `copy_cvref_t<Snd2, Snd>`, let `Shape` be `decltype((shape))` and let `nothrow-callable` be the alias template:

```
template<class... As>  
using nothrow_callable =  
    bool_constant<is_nothrow_invocable_v<decay_t<F>&, Shape,  
    As...>>;
```

1. If any of the types in the *type-list* named by `value_types_of_t<DS, Env, nothrow_callable, type-list>` are `false_type`, then the type of

tag_invoke(get_completion_signatures, snd2, env) shall be **equivalent to**:

```
make_completion_signatures<
  DS, Env, completion_signatures<set_error_t(exception_ptr)>>
```

2. Otherwise, the type of tag_invoke(get_completion_signatures, snd2, env) shall be **equivalent to** completion_signatures_of_t<DS, Env>.

2. ~~If the function selected above~~ Let out_snd be the result of calling bulk(snd, shape, f) or a copy of such. If out_snd is connected to a receiver rcv with environment env such that transform_sender(get-domain-late(out_snd, env), out_snd, env) does not return a sender that invokes f(i, args...) for each i of type Shape from 0 to shape where args is a pack of subexpressions referring to the value completion result datums of the input sender, or does not execute a value completion operation with said datums, the behavior of calling ~~bulk(snd, shape, f)~~ connect(out_snd, rcv) is undefined.

[Editor's note: Change §11.9.6.11 [exec.split] as follows:]

1. split adapts an arbitrary sender into a sender that can be connected multiple times.
2. Let *split-env* be the type of an environment such that, given an instance env, the expression get_stop_token(env) is well-formed and has type stop_token.
3. The name split denotes a customization point object. For some subexpression snd, let Snd be decltype((snd)). If sender_in<Snd, split-env> or constructible_from<decay_t<env_of_t<Snd>>, env_of_t<Snd>> is false, split is ill-formed. Otherwise, the expression split(snd) is expression-equivalent to:

```
transform_sender(
  get-domain-early(snd),
  make_sender(split, snd));
```

1. tag_invoke(split, get_completion_scheduler<set_value_t>(get_env(snd)), snd), if that expression is valid.

— *Mandates*: The type of the tag_invoke expression above satisfies sender.

2. Otherwise, tag_invoke(split, snd), if that expression is valid.

— *Mandates*: The type of the tag_invoke expression above satisfies sender.

1. Let snd be a subexpression such that Snd is decltype((snd)), and let env... be a pack of subexpressions such that sizeof...(env) <= 1 is true. If sender-for<Snd, split t> is false, then the expression split t().transform_sender(snd, env...) is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender snd2 ~~, which that~~:

1. Creates an object sh_state that [Editor's note: ... as before]

[Change §11.9.6.12 [exec.when.all] as follows:]

1. when_all and when_all_with_variant both adapt multiple input senders into a sender that completes when all input senders have completed. when_all only accepts senders with a single value completion signature and on success concatenates all the input senders' value result datums into its own value completion operation. when_all_with_variant(snd...) is semantically equivalent to when_all(into_variant(snd)...), where snd is a pack of subexpressions of sender types.
2. The names when_all and when_all_with_variant denotes ~~a~~ customization point objects. For some subexpressions snd_i..., let Snd_i... be decltype((snd_i)).... The expressions when_all(snd_i...)

~~is~~ and when_all_with_variant(snd_i...) are ill-formed if any of the following is true:

- If the number of subexpressions snd_i... is 0, or
- If any type Snd_i does not satisfy sender.
- If the expression get-domain-early(snd_i) has a different type for any other value of i.

Otherwise, those expressions have the semantics specified below.

[Editor's note: The following paragraph becomes numbered and subsequent paragraphs are renumbered.]

3. ~~Otherwise, the~~The expression when_all(snd_i...) is expression-equivalent to:

```
transform_sender(  
  get-domain-early(snd0),  
  make-when-all-sender(snd0, ... sndn-1))
```

where make-when-all-sender(snd_i...) is expression-equivalent to make-sender(when_all, {}, snd_i...) and returns a sender object w of type w that behaves as follows:

1. tag_invoke(when_all, snd_i...), if that expression is valid. If the function selected by tag_invoke does not return a sender that sends a concatenation of values sent by snd_i... when they all complete with set_value, the behavior of calling when_all(snd_i...) is undefined.

— *Mandates:* The type of the tag_invoke expression above satisfies sender.

2. ~~Otherwise, constructs a sender w of type w:~~ When w is connected with some receiver out_rcv of type outR, it returns an operation state op_state specified as below:

1. For each sender snd_i, ... [Editor's note: ... as before]

...

4. ~~The name when_all_with_variant denotes a customization point object. For some subexpressions snd..., let Snd be decltype((snd)). If any type Snd_i in Snd... does not satisfy sender, when_all_with_variant is ill-formed. Otherwise, the~~ The expression when_all_with_variant(snd_i...) is expression-equivalent to:

```
transform_sender(  
  get-domain-early(snd0),  
  make-sender(when_all_with_variant, {}, snd0, ... sndn-1))
```

1. tag_invoke(when_all_with_variant, snd...), if that expression is valid. If the function selected by tag_invoke does not return a sender that, when connected with a receiver of type Rcv, sends the types into-variant-type<Snd, env_of_t<Rcv>>... when they all complete with set_value, the behavior of calling when_all(snd_i...) is undefined.

— *Mandates:* The type of the tag_invoke expression above satisfies sender.

2. Otherwise, when_all(into_variant(snd)...).

5. Let snd and env be subexpressions such that Snd is decltype((snd)). If sender_for<Snd, when_all_with_variant_t> is false, then the expression when_all_with_variant_t().transform_sender(snd, env) is ill-formed; otherwise, it is equal to:

```
auto [tag, data, ...child] = snd;
return when_all(into_variant(std::move(child))...);
```

[*Note:* This causes the `when_all_with_variant(snd...)` sender to become `when_all(into_variant(snd)...) when it is connected with a receiver whose execution domain does not customize when_all_with_variant. — end note ]`

5. ~~For a sender `snd2` returned from `when_all` or `when_all_with_variant`, `get_env(snd2)` shall return an instance of a class equivalent to `empty_env`.~~ Given a pack of subexpressions `snd...`, let `out_snd` be an object returned from `when_all(snd...)` or `when_all_with_variant(snd...)` or a copy of such, and let `env` be the environment object returned from `get_env(out_snd)`. Given a query object `Q`, `tag_invoke(Q, env)` is expression-equivalent to `get_domain_early(snd0)` when `Q` is `get_domain`; otherwise, it is ill-formed.

[Editor's note: Change §11.9.6.13 [exec.transfer.when.all] as follows:]

1. `transfer_when_all` and `transfer_when_all_with_variant` both adapt multiple input senders into a sender that completes when all input senders have completed, ensuring the input senders complete on the specified scheduler. `transfer_when_all` only accepts senders with a single value completion signature and on success concatenates all the input senders' value result datums into its own value completion operation; `transfer_when_all(scheduler, input-senders...)` is semantically equivalent to `transfer(when_all(input-senders...), scheduler)`. `transfer_when_all_with_variant(scheduler, input-senders...)` is semantically equivalent to `transfer_when_all(scheduler, into_variant(input-senders)...) [ Note: These customizable composite algorithms can allow for more efficient customizations in some cases. — end note ]`
2. The name `transfer_when_all` denotes a customization point object. For some subexpressions `sch` and `snd...`, let `Sch` be `decltype(sch)` and `Snd` be `decltype((snd))`. If `Sch` does not satisfy `scheduler`, or any type `Sndi` in `Snd...` does not satisfy `sender`, `transfer_when_all` is ill-formed. Otherwise, the expression `transfer_when_all(sch, snd...)` is expression-equivalent to:

```
return transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_when_all, sch, snd...));
```

1. `tag_invoke(transfer_when_all, sch, snd...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that sends a concatenation of values sent by `snd...` when they all complete with `set_value`, or does not send its completion operation, other than ones resulting from a scheduling error, on an execution agent belonging to the associated execution resource of `sch`, the behavior of calling `transfer_when_all(sch, snd...)` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, `transfer(when_all(snd...), sch)`.

3. Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))`. If `sender-for<Snd, transfer_when_all_t>` is false, then the expression `transfer_when_all_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```
auto [tag, data, ...child] = snd;
return transfer(when_all(std::move(child)...), std::move(data));
```

[*Note:* This causes the `transfer_when_all(sch, snd...)` sender to become `transfer(when_all(snd...), sch)` when it is connected with a receiver whose execution domain does not customize `transfer_when_all`. — end note]

3. The name `transfer_when_all_with_variant` denotes a customization point object. For some subexpressions `sch` and `snd...`, let `Sch` be `decltype((sch))` and let `Snd` be `decltype((snd))`. If any type `Sndi` in `Snd...` does not satisfy `sender`, `transfer_when_all_with_variant` is ill-formed. Otherwise, the expression `transfer_when_all_with_variant(sch, snd...)` is expression-equivalent to:

```
return transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_when_all_with_variant, sch, snd...));
```

1. `tag_invoke(transfer_when_all_with_variant, snd...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that, when connected with a receiver of type `Rcv`, sends the types `into-variant-type<Snd, env_of_t<Rcv>>...` when they all complete with `set_value`, the behavior of calling `transfer_when_all_with_variant(sch, snd...)` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, `transfer_when_all(sch, into_variant(snd)...)...`.

4. Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))`. If `sender-for<Snd, transfer_when_all_with_variant_t>` is `false`, then the expression `transfer_when_all_with_variant_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```
auto [tag, data, ...child] = snd;
return transfer_when_all(std::move(data),
    into_variant(std::move(child))...);
```

[*Note:* This causes the `transfer_when_all_with_variant(sch, snd...)` sender to become `transfer_when_all(sch, into_variant(snd)...)...` when it is connected with a receiver whose execution domain does not customize `transfer_when_all_with_variant`. — *end note*]

5. For a sender `out_snd` returned from `transfer_when_all(sch, snd...)` or `transfer_when_all_with_variant(sch, snd...)`, `get_env(out_snd)` shall return a queryable object `q` such that `get_domain(q)` shall be expression-equivalent to `get_domain(sch)`, and `get_completion_scheduler<CPO>(q)` returns a copy of `sch`, where `CPO` is either `set_value_t` or `set_stopped_t`. The `get_completion_scheduler<set_error_t>` query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object.

[Editor's note: Change §11.9.6.14 [exec.into.variant] as follows:]

1. `into_variant` adapts a sender with multiple value completion signatures into a sender with just one consisting of a variant of `tuplesnd`.
2. The template `into-variant-type` computes the type sent by a sender returned from `into_variant`.

```
template<class Snd, class Env>
    requires sender_in<Snd, Env>
    using into-variant-type =
        value_types_of_t<Snd, Env>;
```

3. `into_variant` is a customization point object. For some subexpression `snd`, let `Snd` be `decltype((snd))`. If `Snd` does not satisfy `sender`, `into_variant(snd)` is ill-formed. Otherwise, `into_variant(snd)` is expression-equivalent to:

```
transform_sender(
    get-domain-early(snd),
```

```
make-into-variant-sender(snd))
```

where *make-into-variant-sender(snd)* is expression-equivalent to *make-sender(into variant, {}, snd)* and returns a sender object *snd2*; that behaves as follows:

[Editor's note: Reformatting here]

1. When *snd2* is connected with some receiver *out_rcv*, it:
2. Constructs a receiver *rcv*:
 1. If *set_value(rcv, ts...)* is called, calls *set_value(out_rcv, into-variant-type<Snd, env_of_t<decltype((rcv))>>(decayed-tuple<decltype(ts)...>(ts...))*). If this expression throws an exception, calls *set_error(out_rcv, current_exception())*.
 2. *set_error(rcv, err)* is expression-equivalent to *set_error(out_rcv, err)*.
 3. *set_stopped(rcv)* is expression-equivalent to *set_stopped(out_rcv)*.
3. Calls *connect(snd, rcv)*, resulting in an operation state *op_state2*.
4. Returns an operation state *op_state* that contains *op_state2*. When *start(op_state)* is called, calls *start(op_state2)*.
5. Given subexpressions *snd2* and *env [...]* [Editor's note: ...as before]

[Editor's note: Change §11.9.6.15 [exec.stopped.as.optional] as follows:]

1. *stopped_as_optional* maps an input sender's stopped completion operation into the value completion operation as an empty optional. The input sender's value completion operation is also converted into an optional. The result is a sender that never completes with stopped, reporting cancellation by completing with an empty optional.
2. The name *stopped_as_optional* denotes a customization point object. For some subexpression *snd*, let *Snd* be *decltype((snd))*. ~~Let *get-env-sender* be an expression such that, when it is connected with a receiver *rcv*, start on the resulting operation state completes immediately by calling *set_value(rcv, get-env(rcv))*.~~ The expression *stopped_as_optional(snd)* is expression-equivalent to:

```
transform_sender(  
    get-domain-early(snd),  
    make-sender(stopped_as_optional, {}, snd))
```

```
let_value(  
    get-env-sender,  
    [<class Env>(const Env&) requires single-sender<Snd, Env> {  
        return let_stopped(  
            then(snd,  
                [<class T>(T&& t) {  
                    return optional<decay_t<single-sender-value-type<Snd, Env>>>{  
                        std::forward<T>(t)  
                    };  
                }  
            ),  
            [&] () noexcept {  
                return just(optional<decay_t<single-sender-value-type<Snd, Env>>>{  
                    {}  
                });  
            }  
        );  
    }  
);
```

```

}
)

```

- Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))` and `Env` is `decltype((env))`. If either `sender-for<Snd, stopped_as_optional_t>` or `single-sender<Snd, Env>` is false, then the expression `stopped_as_optional_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```

auto [tag, data, child] = snd;
using V = single-sender-value-type<Snd, Env>;
return let_stopped(
    then(std::move(child),
        [<class T>(T&& t) { return optional<V>(std::forward<T>(t)); }],
        []() noexcept { return just(optional<V>()); });

```

[Editor's note: Change §11.9.6.16 [exec.stopped.as.error] as follows:]

- `stopped_as_error` maps an input sender's stopped completion operation into an error completion operation as a custom error type. The result is a sender that never completes with stopped, reporting cancellation by completing with an error.
- The name `stopped_as_error` denotes a customization point object. For some subexpressions `snd` and `err`, let `Snd` be `decltype((snd))` and let `Err` be `decltype((err))`. If the type `Snd` does not satisfy `sender` or if the type `Err` doesn't satisfy `movable-value`, `stopped_as_error(snd, err)` is ill-formed. Otherwise, the expression `stopped_as_error(snd, err)` is expression-equivalent to:

```

let_stopped(snd, [] { return just_error(err); })

```

```

transform_sender(
    get-domain-early(snd),
    make-sender(stopped_as_error, err, snd))

```

- Let `snd` and `env` be subexpressions such that `Snd` is `decltype((snd))` and `Env` is `decltype((env))`. If `sender-for<Snd, stopped_as_error_t>` is false, then the expression `stopped_as_error_t().transform_sender(snd, env)` is ill-formed; otherwise, it is equal to:

```

auto [tag, data, child] = snd;
return let_stopped(
    std::move(child),
    [err = std::move(data)]() mutable { return just_error(std::move(err));
});

```

[Editor's note: Change §11.9.6.17 [exec.ensure.started] as follows:]

- `ensure_started` eagerly starts the execution of a sender, returning a sender that is usable as input to additional sender algorithms.
- Let `ensure-started-env` be the type of an execution environment such that, given an instance `env`, the expression `get_stop_token(env)` is well-formed and has type `stop_token`.
- The name `ensure_started` denotes a customization point object. For some subexpression `snd`, let `Snd` be `decltype((snd))`. If `sender_in<Snd, ensure-started-env>` or `constructible_from<decay_t<env_of_t<Snd>>, env_of_t<Snd>>` is false, `ensure_started(snd)` is ill-formed. Otherwise, the expression `ensure_started(snd)` is expression-equivalent to:

```

transform_sender(
    get-domain-early(snd),
    make-sender(ensure_started, {}, snd));

```

1. `tag_invoke(ensure_started, get_completion_scheduler<set_value_t>(get_env(snd)), snd)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(ensure_started, snd)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

1. Let `snd` be a subexpression such that `Snd` is `decltype((snd))`, and let `env...` be a pack of subexpressions such that `sizeof...(env) <= 1` is true. If `sender_for<Snd, ensure_started_t>` is false, then the expression `ensure_started_t().transform_sender(snd, env...)` is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender `snd2`, ~~which~~ that:

1. Creates an object `sh_state` that [Editor's note: ... as before]

[Editor's note: Change §11.9.7.1 [exec.start.detached] as follows:]

1. `start_detached` eagerly starts a sender without the caller needing to manage the lifetimes of any objects.

2. The name `start_detached` denotes a customization point object. For some subexpression `snd`, let `Snd` be `decltype((snd))`. ~~If `Snd` does not satisfy sender~~ If `sender_in<Snd, empty_env>` is false, `start_detached` is ill-formed. Otherwise, the expression `start_detached(snd)` is expression-equivalent to:

```
apply_sender(get_domain_early(snd), start_detached, snd)
```

— *Mandates*: The type of the expression above is `void`.

If the ~~function-selected~~ expression above does not eagerly start the sender `snd` after connecting it with a receiver that ignores value and stopped completion operations and calls `terminate()` on error completions, the behavior of calling `start_detached(snd)` is undefined.

1. `tag_invoke(start_detached, get_completion_scheduler<set_value_t>(get_env(snd)), snd)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above is `void`.

2. Otherwise, `tag_invoke(start_detached, snd)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above is `void`.

3. Otherwise, let `Rcv` be the type of a receiver, let `rcv` be an rvalue of type `Rcv`, and let `crcv` be a lvalue reference to `const Rcv` such that:

— The expression `set_value(rcv)` is not potentially-throwing and has no effect,

— For any subexpression `err`, the expression `set_error(rcv, err)` is expression-equivalent to `terminate()`,

— The expression `set_stopped(rcv)` is not potentially-throwing and has no effect, and

— The expression `get_env(crcv)` is expression-equivalent to `empty_env{}`.

Calls `connect(snd, rcv)`, resulting in an operation state `op_state`, then calls `start(op_state)`.

3. Let `snd` be a subexpression such that `Snd` is `decltype((snd))`, and let *detached-receiver* and *detached-operation* be the following exposition-only class types:

```
struct detached-receiver {
    using is_receiver = unspecified;
    detached-operation* op; // exposition only

    friend void tag_invoke(set_value_t, detached-receiver&& self) noexcept {
        delete self.op; }
    friend void tag_invoke(set_error_t, detached-receiver&&, auto&&)
        noexcept { terminate(); }
    friend void tag_invoke(set_stopped_t, detached-receiver&& self) noexcept
        { delete self.op; }
    friend empty_env tag_invoke(get_env_t, const detached-receiver&)
        noexcept { return {}; }
};

struct detached-operation {
    connect_result_t<Snd, detached-receiver> op; // exposition only

    explicit detached-operation(Snd&& snd)
        : op(connect(std::forward<Snd>(snd), detached-receiver{this}))
    {}
};
```

If `sender_to<Snd, detached-receiver>` is false, then the expression `start_detached_t().apply_sender(snd)` is ill-formed; otherwise, it is expression-equivalent to:

```
start((new detached-operation(snd))->op)
```

[Editor's note: Change §11.9.7.2 [exec.sync.wait] as follows:]

1. [...]

4. The name `this_thread::sync_wait` denotes a customization point object. For some subexpression `snd`, let `Snd` be `decltype((snd))`. If `sender_in<Snd, sync-wait-env>` is false, or ~~the number of the arguments completion signatures of `t<Snd, sync-wait-env>::value_types` passed into the variant-template parameter is not 1 if the type completion signatures of `t<Snd, sync-wait-env, type-list, type identity t>` is ill-formed~~, `this_thread::sync_wait(snd)` is ill-formed. Otherwise, `this_thread::sync_wait(snd)` is expression-equivalent to:

```
apply_sender(get-domain-early(snd), sync_wait, snd)
```

— *Mandates:* The type of expression above is *sync-wait-type<Snd, sync-wait-env>*.

1. `tag_invoke(this_thread::sync_wait, get_completion_scheduler<set_value_t>(get_env(snd)), snd)`, if this expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-type<Snd, sync-wait-env>*.

2. Otherwise, `tag_invoke(this_thread::sync_wait, snd)`, if this expression is valid and its type is.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-type<Snd, sync-wait-env>*.

5. ~~Otherwise:~~ Let *sync-wait-receiver* be a class type that satisfies *receiver*, let `rcv` be an xvalue of that type, and let `crcv` be a const lvalue referring to `rcv` such that `get_env(crcv)` has type *sync-wait-env*. If `sender_in<Snd, sync-wait-env>` is false, or if the type completion signatures of `t<Snd, sync-wait-env, type-list, type identity t>` is ill-

formed, the expression `sync wait t().apply_sender(snd)` is ill-formed; otherwise, it has the following effects:

1. ~~Constructs a receiver `rcv`:~~

2. Calls `connect(snd, rcv)`, resulting in an operation state `op_state`, then calls `start(op_state)`.

3. Blocks the current thread until a completion operation of `rcv` is executed. When it is:

1. If `set_value(rcv, ts...)` has been called, returns *sync-wait-type*<`Snd`, *sync-wait-env*>{*decayed-tuple*<`decltype(ts)...`>{`ts...`}*}*. If that expression exits exceptionally, the exception is propagated to the caller of `sync_wait`.

2. If `set_error(rcv, err)` has been called, let `Err` be the decayed type of `err`. If `Err` is `exception_ptr`, calls `std::rethrow_exception(err)`. Otherwise, if the `Err` is `error_code`, throws `system_error(err)`. Otherwise, throws `err`.

3. If `set_stopped(rcv)` has been called, returns *sync-wait-type*<`Snd`, *sync-wait-env*>{*}*.

6. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For some subexpression `snd`, let `Snd` be the type of `into_variant(snd)`. If `sender_in<Snd, sync-wait-env>` is `false`, `this_thread::sync_wait_with_variant(snd)` is ill-formed. Otherwise, `this_thread::sync_wait_with_variant(snd)` is expression-equivalent to:

```
apply_sender(get_domain_early(snd), sync_wait_with_variant, snd)
```

— *Mandates:* The type of expression above is *sync-wait-with-variant-type*<`Snd`, *sync-wait-env*>.

1. `tag_invoke(this_thread::sync_wait_with_variant, get_completion_scheduler<set_value_t>(get_env(snd)), snd)`, if this expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-with-variant-type*<`Snd`, *sync-wait-env*>.

2. Otherwise, `tag_invoke(this_thread::sync_wait_with_variant, snd)`, if this expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-with-variant-type*<`Snd`, *sync-wait-env*>.

6. ~~Otherwise,~~The expression `sync wait with variant t().apply_sender(snd)` is expression-equivalent to `this_thread::sync_wait(into_variant(snd))`.

[Update §11.10 [exec.execute] as follows:]

1. `execute` creates fire-and-forget tasks on a specified scheduler.

2. The name `execute` denotes a customization point object. For some subexpressions `sch` and `f`, let `Sch` be `decltype((sch))` and `F` be `decltype((f))`. If `Sch` does not satisfy `scheduler` or `F` does not satisfy `invocable`, `execute` is ill-formed. Otherwise, `execute` is expression-equivalent to:

```
apply_sender(
    query_or_default(get_domain, sch, default_domain()),
    execute, schedule(sch), f)
```

— *Mandates:* The type of the expression above is `void`.

1. `tag_invoke(execute, sch, f)`, if that expression is valid. If the function selected by `tag_invoke` does not invoke the function `f` (or an object decay-copied from `f`) on an execution agent belonging to the associated execution resource of `sch`, or if it does not call `std::terminate` if an error occurs after control is returned to the caller, the behavior of calling `execute` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above is `void`.

3. Otherwise, For some subexpressions `snd` and `f` where `F` is `decltype((f))`, if `F` does not satisfy invocable, the expression `execute t().apply_sender(snd, f)` is ill-formed; otherwise, it is expression-equivalent to `start_detached(then(schedule(sch)snd, f))`.

§ 5 References

[P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r5>

[P2141R1] Antony Polukhin. 2023-05-03. Aggregates are named tuples.

<https://wg21.link/p2141r1>

[P2300R7] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2023-04-21. ``std::execution``.

<https://wg21.link/p2300r7>

[stdexecgithub] stdexec.

<https://github.com/NVIDIA/stdexec>